HDOC Day-46

Name- Aaryan Pratap Singh

Sap id - 590032842

Algorithm

Main Algorithm

1. Start.
2. Read the size `n` of the array.
3. Validate that `n` is positive.
4. Declare a variable-length integer array `nums[n]`.
5. Read `n` elements into `nums`.
6. Pass the address of `nums` to `Bubble_sort()`.
7. In `Bubble_sort()`:
   - Compare adjacent elements.
   - Swap them if the left element is greater than the right element.
   - Repeat until the array is sorted in ascending order.
8. Display the sorted array.
9. Pass the sorted array to `occurrence()`.
10. In `occurrence()`:
    - Read `target`.
    - Initialize `first = -1` and `last = -1`.
    - Traverse the sorted array.
    - When `target` is found for the first time, store its index in `first`.
    - Continue searching and update `last` whenever `target` occurs.
    - If `first` remains `-1`, display `-1, -1`.
    - Otherwise, display the value and index of the first and last occurrences.
11. Stop.

Bubble Sort Concept

Compare 5 and 2 → swap
Pass 1Comparison 1 of 61 swap
2
5
7
1
6
3
4
1
2
3
4
5

6
7
Result: 2, 5; 5 moves one place right
Sorting step

ShufflePlayStepFinish pass
Give feedback

C Program

```
#include <stdio.h>



void Bubble_sort(int *nums, int size);

void occurrence(int *nums, int size);



int main()

{

   int size, i;



   /* Read and validate array size */

   do

   {

      printf("Enter the size of the array: ");

      scanf("%d", &size);



      if (size <= 0)

      {

         printf("Invalid size. Size must be positive.\n");

      }
```

```
} while (size <= 0);

/* Variable-length array */
int nums[size];

printf("Enter %d array elements:\n", size);

for (i = 0; i < size; i++)
{
    scanf("%d", nums + i);
}

/* Pass array using its address */
Bubble_sort(nums, size);

printf("\nSorted array: ");

for (i = 0; i < size; i++)
{
    printf("%d ", *(nums + i));
}

printf("\n");
```

```
    /* Pass sorted array to occurrence() */
    occurrence(nums, size);

    return 0;
}

/* Function to sort the array using Bubble Sort */
void Bubble_sort(int *nums, int size)
{
    int i, j, temp;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(nums + j) > *(nums + j + 1))
            {
                temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
                *(nums + j + 1) = temp;
            }
        }
    }
```

```
}

/* Function to find first and last occurrences */
void occurrence(int *nums, int size)
{
    int target;
    int first = -1;
    int last = -1;
    int i;

    printf("\nEnter the target: ");
    scanf("%d", &target);

    for (i = 0; i < size; i++)
    {
        if (*(nums + i) == target)
        {
            if (first == -1)
            {
                first = i;
            }

            last = i;
        }
```

```
    }
    if (first == -1)
    {
        printf("-1, -1\n");
    }
    else
            printf("First occurrence: %d at index %d\n",
              *(nums + first), first);
   printf("Last occurrence: %d at index %d\n",
              *(nums + last), last);
    }
}
```

Test-Case Table

Test	Input Array	Target	Sorted Array	Expected Result
1	`5 7 7 8 8 10`	8	`5 7 7 8 8 10`	First: 8 at index 3, Last: 8 at index 4
2	`5 7 7 8 10`	8	`5 7 7 8 10`	First: 8 at index 3, Last: 8 at index 3
3	`5 7 7 8 8 10`	6	`5 7 7 8 8 10`	`-1, -1`
4	`8 2 8 5 1 8`	8	`1 2 5 8 8 8`	First: 8 at index 3, Last: 8 at index 5
5	`4 4 4 4`	4	`4 4 4 4`	First: 4 at index 0, Last: 4 at index 3
6	`10`	10	`10`	First: 10 at index 0, Last: 10 at index 0
7	`10`	5	`10`	`-1, -1`
8	Size = `0`	—	—	Invalid size; asks again

Execution

```
[aaryanrajput@aaryans-macbook ~ % nano day46.c
[aaryanrajput@aaryans-macbook ~ % clang day46.c -o day46.c
[aaryanrajput@aaryans-macbook ~ % ./day46.c
Enter the size of the array: 6
Enter 6 array elements:
5 7 7 8 8 10

Sorted array: 5 7 7 8 8 10

Enter the target: 7
First occurrence: 7 at index 1
Last occurrence: 7 at index 2
aaryanrajput@aaryans-macbook ~ %
```